

JAVASCRIPT

Interview Cheatsheet — Clean 2–3 Line Definitions

Core Language · Async · Scope & Closures · Prototypes · Modern ES Features

What's inside

1. Scope, Closures & Execution Context
2. this, Functions & Binding
3. Prototypes & OOP
4. Async JS — Callbacks, Promises, Async/Await
5. Event Loop & Concurrency
6. Arrays, Objects & Iteration
7. ES6+ Features
8. Modules & Memory
9. Browser & DOM Concepts
10. Performance & Design Patterns

1. Scope, Closures & Execution Context

Execution Context

The environment in which JS code runs, containing variable bindings, scope chain, and the value of `this`. A new context is created for global code and each function call.

Call Stack

A LIFO structure that tracks function calls in execution. When a function is invoked, it's pushed; when it returns, it's popped — a full stack causes a "stack overflow".

Lexical Scope

Scope determined by where variables/functions are physically written in the code, not where they're called from. Inner functions have access to outer function variables.

Closure

A function that retains access to its outer (lexical) scope's variables even after the outer function has returned. Commonly used for data privacy and factory functions.

var vs let vs const

`var` is function-scoped and hoisted with `undefined`; `let` / `const` are block-scoped and live in a "temporal dead zone" until initialized. `const` disallows reassignment.

Hoisting

JS moves variable and function declarations to the top of their scope during compilation. `var` and function declarations are hoisted and initialized; `let` / `const` are hoisted but not initialized.

Temporal Dead Zone (TDZ)

The period between entering a scope and a `let` / `const` variable's declaration being executed, during which accessing the variable throws a `ReferenceError`.

IIFE

An Immediately Invoked Function Expression — a function defined and executed right away, historically used to create a private scope before block scoping existed.

2. this, Functions & Binding

this keyword

A reference whose value depends on how a function is called: the object before the dot for method calls, the global/undefined for plain calls, or the new instance for constructors.

call() / apply() / bind()

`call` and `apply` invoke a function immediately with a given `this` (args as list vs array); `bind` returns a new function permanently bound to a given `this`.

Arrow Functions

Functions that don't have their own `this`, `arguments`, or `super` — they inherit these lexically from the enclosing scope, making them ideal for callbacks that need outer context.

Function Declaration vs Expression

Declarations (`function foo(){}`) are hoisted fully and callable before their definition; expressions (`const foo = function(){}`) are not callable until the assignment runs.

Higher-Order Function

A function that takes another function as an argument or returns a function, enabling patterns like `map`, `filter`, decorators, and middleware.

Currying

Transforming a function that takes multiple arguments into a sequence of functions each taking a single argument, enabling partial application and reusable configured functions.

Default & Rest Parameters

Default parameters (`function f(a=1)`) provide fallback values; rest parameters (`...args`) collect remaining arguments into an array.

arguments object

An array-like object available inside regular (non-arrow) functions containing all passed arguments, largely superseded by rest parameters in modern code.

3. Prototypes & OOP

Prototype

Every JS object has an internal link (`[[Prototype]]`) to another object it inherits properties/methods from, forming the prototype chain used for property lookup.

Prototype Chain

The chain of linked prototype objects JS walks up when a property isn't found on an object directly, ending at `Object.prototype` and finally `null`.

__proto__ vs prototype

`__proto__` is the actual link on an instance to its prototype; `prototype` is a property on constructor functions used to set that link for instances they create.

ES6 Class

Syntactic sugar over prototype-based inheritance, providing a cleaner `class` / `extends` / `constructor` syntax while still using prototypes under the hood.

Object.create()

Creates a new object with a specified prototype object, allowing explicit prototypal inheritance without invoking a constructor function.

Getters & Setters

Special methods (`get` / `set`) that let you define computed properties, intercepting reads and writes to an object property with custom logic.

Private Class Fields

Fields prefixed with `#` (e.g. `#balance`) that are truly private to a class, inaccessible from outside — unlike the older convention of prefixing with underscore.

Mixins

A pattern for composing behavior into classes by copying methods from multiple source objects, used as an alternative to single-inheritance limitations.

4. Async JS — Callbacks, Promises, Async/Await

Callback

A function passed as an argument to be executed later, typically after an async operation completes — prone to deeply nested "callback hell" for complex flows.

Promise

An object representing the eventual result of an async operation, existing in one of three states: pending, fulfilled, or rejected — consumed via `.then().catch()`.

Promise.all / allSettled

`Promise.all` resolves when every promise resolves (or rejects on first failure); `allSettled` waits for all to settle and returns each result's status regardless of success/failure.

Promise.race / any

`Promise.race` settles as soon as the first promise settles (success or failure); `Promise.any` resolves on the first fulfillment, ignoring rejections unless all reject.

async / await

Syntax that lets asynchronous, Promise-based code be written in a synchronous-looking style. `await` pauses function execution until the awaited Promise settles.

try/catch with async

Wraps `await` calls to handle Promise rejections synchronously-looking, replacing chained `.catch()` calls with standard error handling syntax.

Microtasks vs Macrotasks

Microtasks (Promise callbacks, `queueMicrotask`) run before the next render/macrotask; macrotasks (`setTimeout`, I/O) run after all queued microtasks have been drained.

Generators

Functions defined with `function*` that can pause and resume execution using `yield`, producing an iterator — a foundation for some async patterns and iterables.

5. Event Loop & Concurrency

Event Loop

The mechanism that continuously checks if the call stack is empty and, if so, pushes the next queued callback (from the microtask or macrotask queue) onto it.

Web APIs / Node APIs

Browser or Node-provided async capabilities (timers, DOM events, fetch, fs) that run outside the single JS thread, queuing their callbacks once complete.

setTimeout / setInterval

`setTimeout` queues a callback to run once after a minimum delay; `setInterval` repeats it on a fixed cadence — both are macrotasks, not guaranteed to run exactly on time.

Single-Threaded Nature

JS executes on one main thread, meaning long-running synchronous code blocks everything else (rendering, events) until it completes — a key reason to favor async patterns.

Web Workers

A browser API that runs scripts on a separate background thread, enabling true parallelism for CPU-heavy tasks without blocking the main thread.

Debounce vs Throttle

Debounce delays execution until a pause in events (e.g. after typing stops); throttle limits execution to at most once per fixed interval regardless of event frequency.

6. Arrays, Objects & Iteration

map / filter / reduce

`map` transforms each element into a new array; `filter` keeps elements matching a condition; `reduce` accumulates all elements into a single value.

Spread Operator

`...` expands an iterable's elements or an object's properties in place, commonly used for shallow copying, merging, and passing array elements as arguments.

Destructuring

Syntax to unpack values from arrays or properties from objects into distinct variables in one statement, supporting defaults and renaming.

Shallow vs Deep Copy

A shallow copy duplicates only the top-level structure (nested objects still shared by reference); a deep copy recursively duplicates all nested structures independently.

for...in vs for...of

`for...in` iterates over an object's enumerable property keys (including inherited ones); `for...of` iterates over the values of an iterable like arrays, strings, or Maps.

Map & Set

`Map` stores key-value pairs with keys of any type and predictable iteration order; `Set` stores unique values only, useful for deduplication.

WeakMap & WeakSet

Like Map/Set but hold weak references to their keys/values, allowing garbage collection when no other references exist — useful for metadata without memory leaks.

Object.freeze / Object.seal

`freeze` prevents any changes to an object (add, remove, modify); `seal` prevents adding/removing properties but still allows modifying existing ones.

7. ES6+ Features

Template Literals

Strings wrapped in backticks supporting embedded expressions via `${}` and multi-line text without explicit concatenation or escape characters.

Optional Chaining (?.)

Safely accesses deeply nested properties, short-circuiting to `undefined` instead of throwing if any part of the chain is `null` / `undefined`.

Nullish Coalescing (??)

Returns the right-hand value only when the left is `null` or `undefined` — unlike `||`, it doesn't fall through for falsy-but-valid values like `0` or `''`.

Symbol

A primitive type representing a unique, immutable identifier, often used as a non-colliding object property key or to define custom iterator behavior.

Iterators & Iterables

An iterable implements `Symbol.iterator`, returning an iterator object with a `next()` method — the protocol behind `for...of`, spread, and destructuring of collections.

Tagged Templates

A function called with a template literal's string parts and interpolated values passed separately, enabling custom string processing like sanitization or i18n.

BigInt

A numeric primitive for integers beyond `Number.MAX_SAFE_INTEGER`, created by appending `n` to a literal, e.g. `123n`.

Proxy & Reflect

`Proxy` wraps an object to intercept and customize fundamental operations (get, set, delete); `Reflect` provides matching methods to perform those default operations explicitly.

8. Modules & Memory

ES Modules (import/export)

The native module system using `import` / `export` statements, statically analyzed at build time and enabling tree-shaking of unused code.

CommonJS (require/module.exports)

Node's traditional module system, synchronous and dynamically resolved at runtime — still common in older Node codebases and some tooling.

Garbage Collection

JS engines automatically reclaim memory for objects no longer reachable from any root reference, primarily using a mark-and-sweep algorithm.

Memory Leak

Occurs when references to unused objects are unintentionally kept alive — common causes include forgotten timers, detached DOM nodes, and uncleared event listeners.

Strict Mode

`'use strict'` opts into a restricted variant of JS that throws errors for common mistakes (undeclared variables, silent failures) and disables some unsafe features.

9. Browser & DOM Concepts

Event Bubbling & Capturing

Bubbling propagates an event from the target element up through its ancestors; capturing propagates it downward from the root first — controlled via the listener's `capture` option.

Event Delegation

Attaching a single listener to a parent element to handle events from its children (via bubbling), reducing the number of listeners needed for dynamic lists.

preventDefault / stopPropagation

`preventDefault` stops a browser's default action (like a link navigating); `stopPropagation` stops the event from continuing to bubble/capture further.

localStorage / sessionStorage

Browser key-value storage APIs; `localStorage` persists across sessions with no expiry, `sessionStorage` is cleared when the tab closes. Both store strings only.

Fetch API

A Promise-based API for making HTTP requests, replacing `XMLHttpRequest` with a cleaner interface for handling responses, headers, and streaming bodies.

CORS

A browser security mechanism that restricts cross-origin HTTP requests unless the server explicitly allows it via response headers like `Access-Control-Allow-Origin`.

10. Performance & Design Patterns

Module Pattern

Uses closures (often via an IIFE) to create private state and expose only a public API, emulating encapsulation before ES modules existed.

Singleton Pattern

Ensures a class or module has only one instance and provides a single global point of access to it, often used for shared state like config or caches.

Observer Pattern

An object (subject) maintains a list of dependents (observers) and notifies them automatically of state changes — the conceptual basis for events and reactive streams.

Memoization

Caching the result of an expensive function call keyed by its arguments, so repeated calls with the same input return the cached result instantly.

Lazy Evaluation

Delaying computation of a value until it's actually needed, reducing unnecessary work — seen in generators, dynamic imports, and short-circuit operators.

Web Vitals (perf metrics)

Key user-centric performance metrics like LCP (loading), INP (interactivity), and CLS (visual stability) used to measure real-world page experience.

Prepared as a quick-revision reference for JavaScript interview prep — covers core language mechanics through modern ES features and browser APIs.